Cerberus: A Multi-Agent Orchestration Framework for the Detection and Repair of Security Vulnerabilities in LLM-Generated Code

April 1, 2026

Team Members

Anirudh S Nair	TVE23CS037
Navya M J	TVE23CS100
Niharika S	TVE23CS105
Shravan Nander Pandala	TVE23CS131

Project Guide

Prof. Bincy P Mathew
Assistant Professor
Department of Computer Science and Engineering
College of Engineering Trivandrum

Contents I

1. Introduction and Problem Context
2. Motivation & Literature Review
3. Motivation & Literature Review
4. Problem Statement
5. Functionalities
6. System Architectural Diagram
7. Module Description
8. Data Flow Diagram
9. Work Plan

Contents II

10 Software & Hardware Framework:

11 Results

12 Conclusion

13 References

1. Introduction to the Problem Area and Context

Application Domain

- DevSecOps (Development, Security, and Operations)
- AI-Assisted Software Development and Automated Vulnerability Remediation

Difficulties of a Manual/Traditional Solution

- Traditional static analysis tools output massive, dense text reports.
- Developers must manually read the report, locate the code, research the proper patch, and rewrite the logic.
- This process is slow, prone to human error, and disrupts the developer's workflow.

Why a Computerized/Digital Solution is Required?

- An automated solution eliminates the gap between detecting a vulnerability and writing the fix.
- AI agents provide context-aware, instantaneous remediation directly inside the IDE.

2.1 Motivation & Literature Survey

Existing Work	Merits	Demerits
Security Degradation in Iterative AI Code Generation (2025)	Shows iterative prompts improve readability but may reduce security; provides empirical analysis of vulnerabilities in AI-generated code.	Focuses mainly on identifying the issue and does not propose an automated repair mechanism.
SecureFixAgent: Hybrid LLM Agent for Vulnerability Repair (2025)	Combines vulnerability localization tools with LLM repair, improving accuracy and reducing hallucinations.	Reliance on localized, single-file context and lightweight models.
AI-driven Security as Code using Multi-Agent Systems (2025)	Uses specialized agents (Attacker, Defender, Auditor) to collaboratively analyze and secure systems.	Inability to provide cross-file architectural repairs.
RL for Automated Penetration Testing (2025)	Uses DRL agents to autonomously discover vulnerabilities and simulate attack paths.	Requires long training time and complex reward function design.
LLM Critics Help Catch LLM Bugs (2024)	AI critic models detect bugs in LLM-generated code and assist human review.	Produces false positives by hallucinating vulnerabilities.

2.2 Motivation & Literature Survey

Existing Solutions

- **Static Security Analyzers (e.g., Bandit, SonarQube)**
  - **Features:** Fast, rule-based vulnerability detection with high accuracy.
  - **Drawbacks:** No automated remediation; developers must manually patch issues.

- **Standard AI Copilots**
  - **Features:** Fast code generation and seamless IDE integration.
  - **Drawbacks:** Require manual prompting and may hallucinate insecure code without strict constraints.

Motivation

- Existing tools either **detect vulnerabilities without fixing them** or **generate code without strong security guarantees**.
- This creates a need for systems like **Cerberus**, which combine **vulnerability detection with automated AI-driven remediation**.

3. Problem Statement

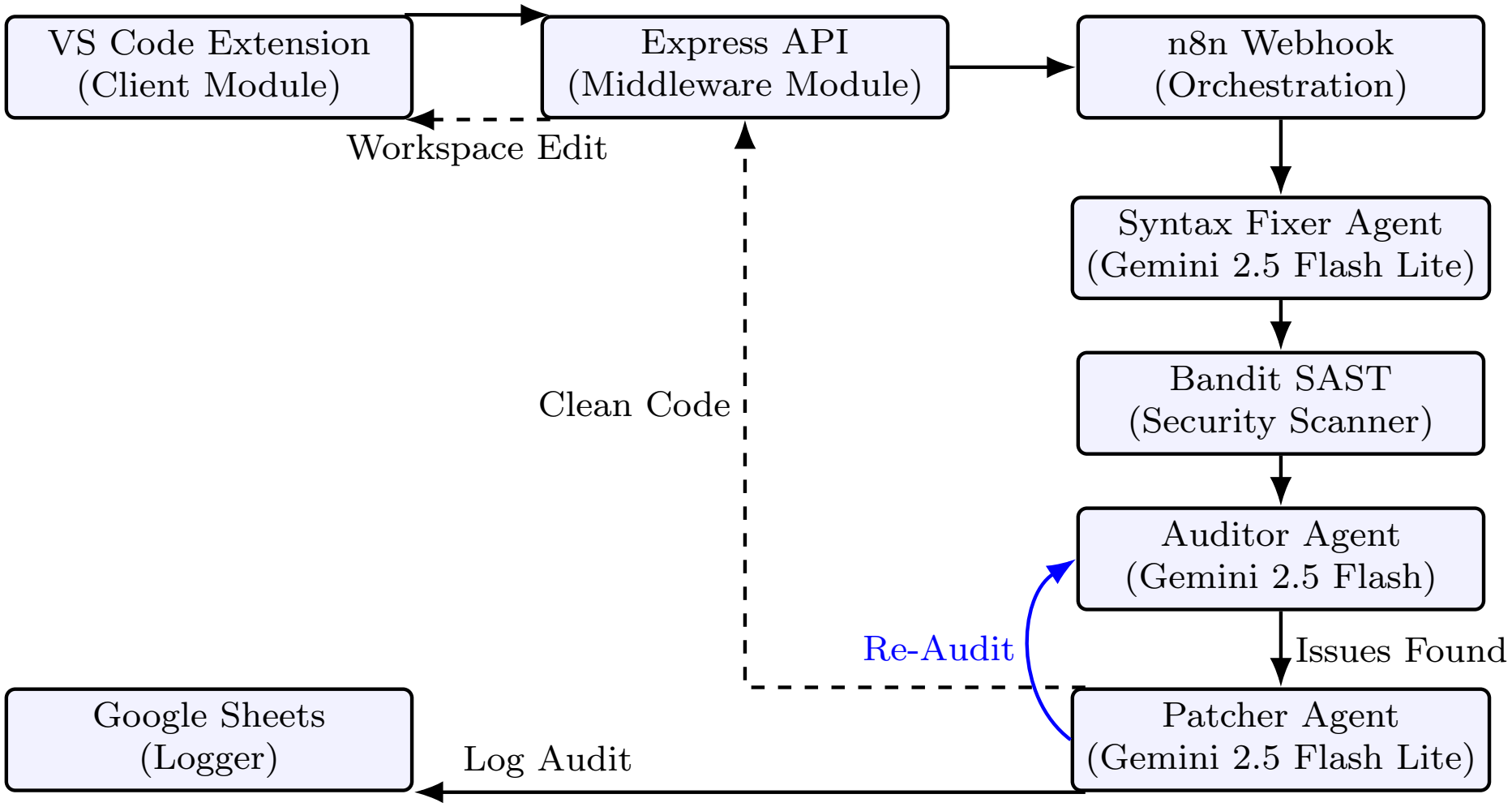
The problem is stated as to how the rapid adoption of AI-driven development has introduced a critical security gap where increased coding velocity often comes at the cost of software integrity. This problem is compounded by the limitations of current security tools, which primarily offer passive detection rather than active remediation. Without a deterministic system to verify that AI-generated fixes are actually secure, organizations remain vulnerable to probabilistic errors and the introduction of unverified flaws into their production environments.

4. Functionalities

The system will provide the following core functionalities:

1. **Extract** the active Python code from the developer's VS Code editor environment.
2. **Transmit** the code securely to a local Express middleware API.
3. **Scan** the code for vulnerabilities using the Bandit security tool within an n8n workflow.
4. **Generate** a highly structured JSON report detailing the vulnerabilities using an Auditor AI Agent.
5. **Rewrite** the vulnerable code safely using a specialized Patcher AI Agent.
6. **Replace** the original text in the VS Code editor with the patched, secure code.

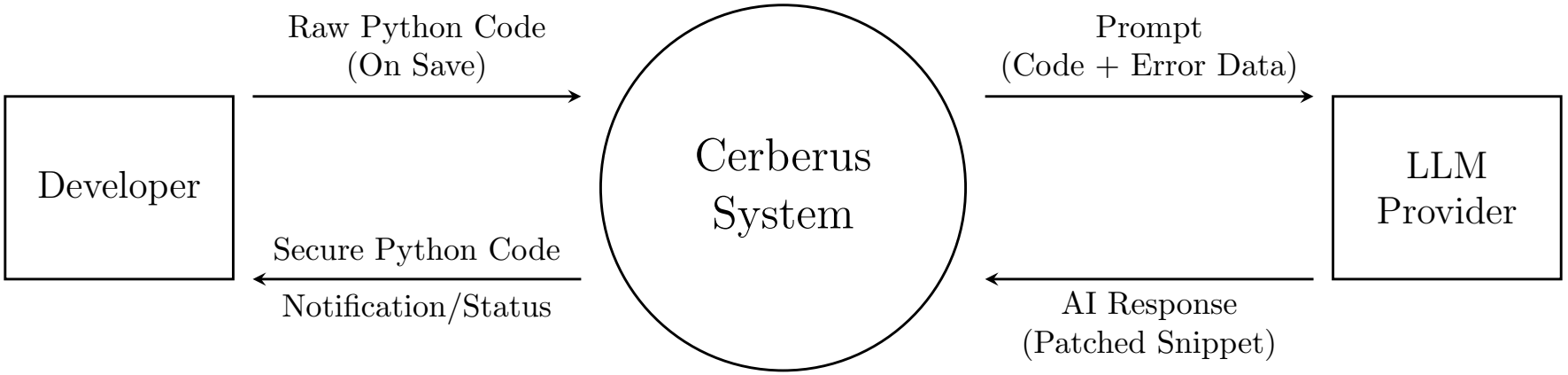
5. System Architectural Diagram



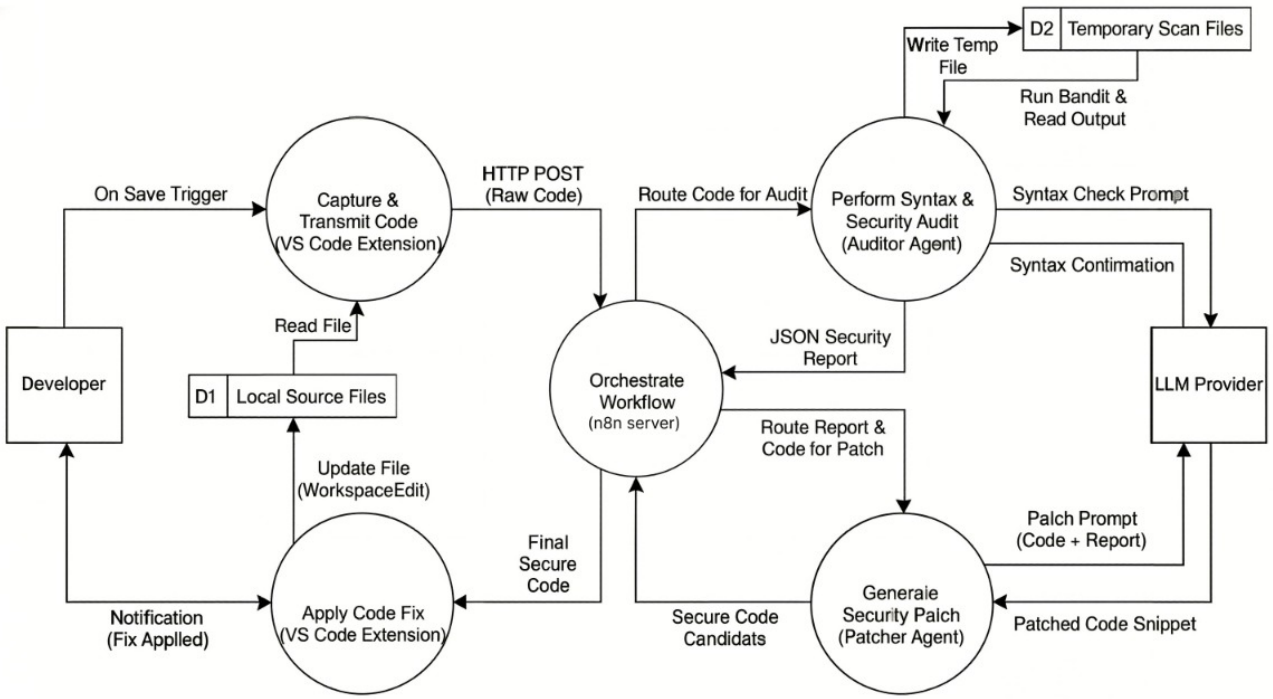
6. Module Description

Module Name	Input	Processing	Output
VS Code Extension	User Command	Extracts active editor text & transmits JSON payload	Safely overwrites editor text
Express API Middleware	JSON Payload	Node.js proxy to cloud n8n webhook, handles CORS & timeouts	Routes response to IDE
Syntax Fixer Agent	Raw Python Code	Validates and corrects structural formatting without altering logic	Parseable Python Code
Auditor Agent	Validated Code	Executes Bandit SAST scan & maps issues via LLM reasoning	Strict JSON Audit Report
Patcher Agent	Report & Code	Employs specialized coding LLM to safely rewrite vulnerable logic	Secured Python Code

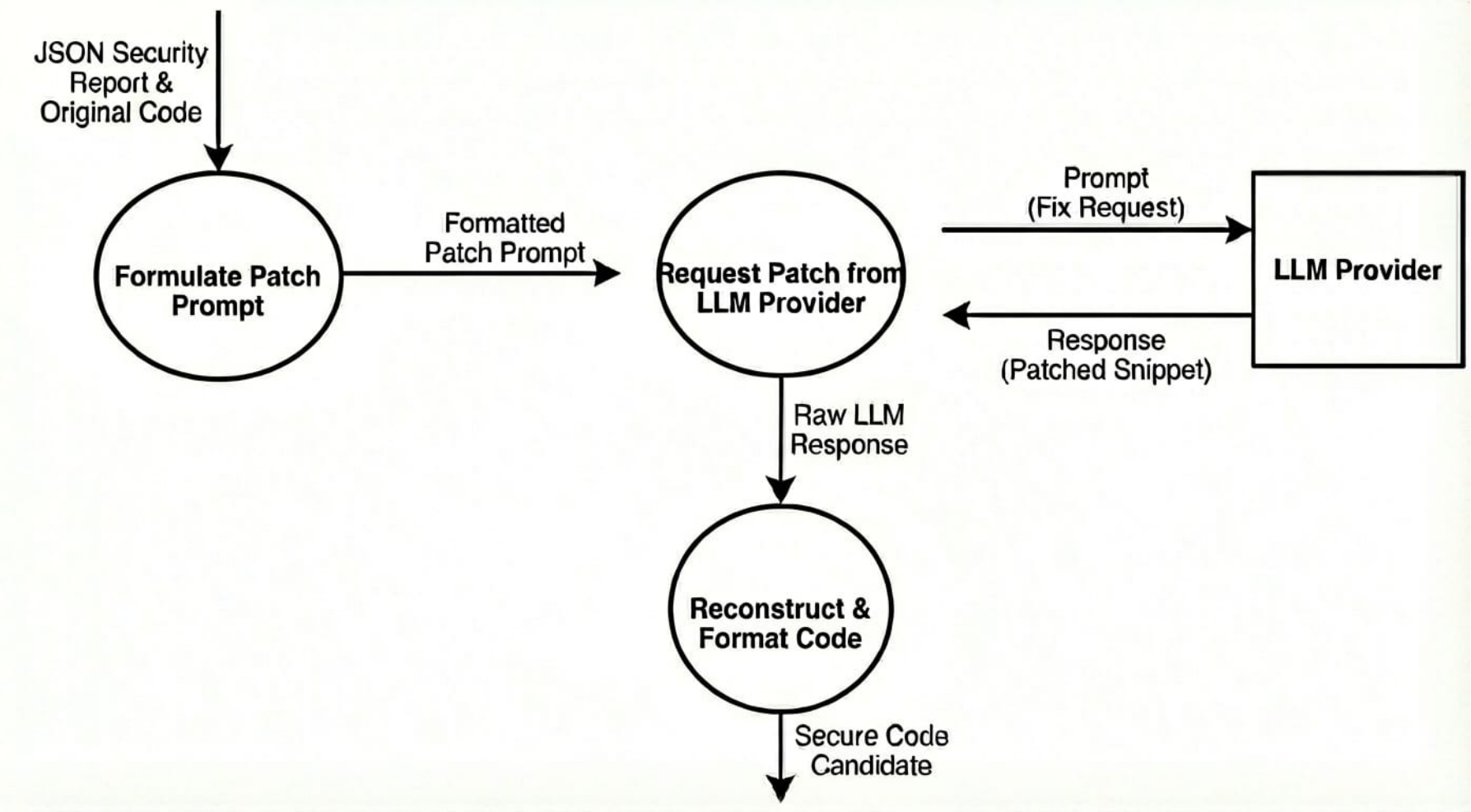
7. Data Flow Diagram: Level 0



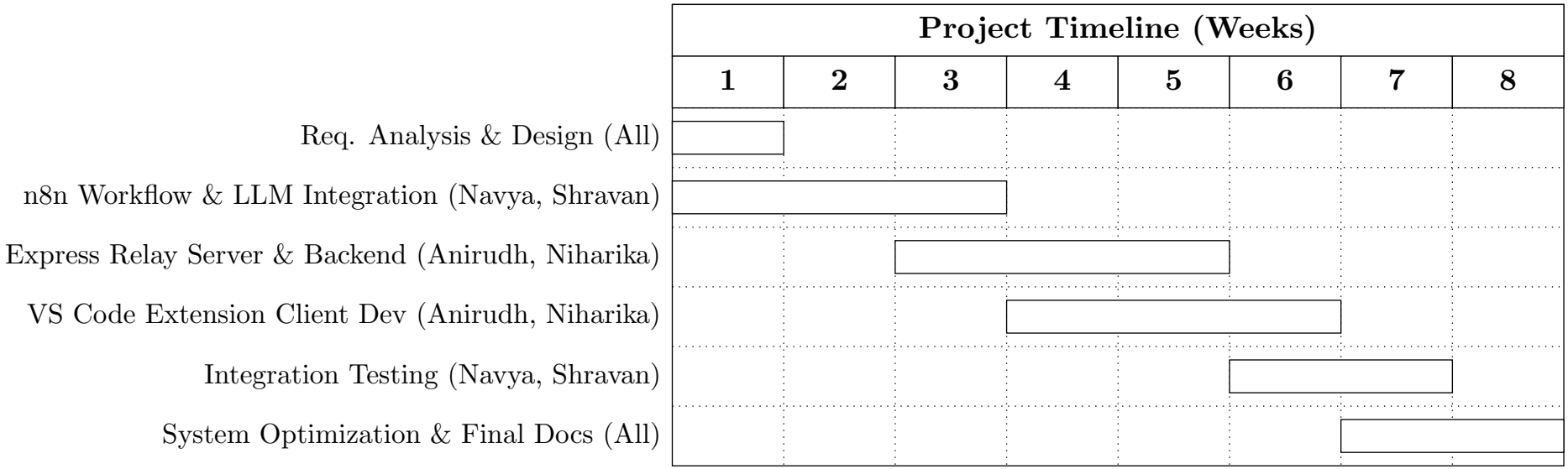
7. Data Flow Diagram: Level 1



7. Data Flow Diagram: Level 2 (Detailed Decomposition of Generation of Security Patch)



8.1 Work Plan: Timeline & Module Allocation



Division of Module-Based Work

- **Anirudh & Niharika:** VS Code Extension (Tree View, Real-Time Scanner, Patch Engine) and Express Relay Server (API, Rate-Limiting, Diff Analyser).
- **Navya & Shravan:** n8n Agentic Workflow (Auditor & Patcher Agents), LLM Integration, and Integration Testing & System Optimisation.

8.2 Work Plan: Adherence & Project Execution

Project Status: **Fully Completed**

Milestones Achieved (Weeks 1–8):

- **Analysis & Design (W1):** Defined multi-agent architecture and client-backend communication protocols.
- **Workflow & LLM (W1–4):** Integrated Auditor, Patcher, and Syntax Fixer agents into the n8n environment for automated remediation.
- **Backend & Relay (W3–5):** Deployed Express.js middleware to manage asynchronous requests and bypass timeout constraints.
- **Client Development (W4–6):** Finalized VS Code UI and the iterative `autoPatchRemainingLoops` logic.
- **Testing & Optimization (W6–8):** Verified self-healing against vulnerabilities and optimized system response latency.

Justification of Success:

- Adherence to the Gantt chart was maintained by parallelizing AI workflow and extension development. This modular approach ensured ample time for final phase testing and documentation.

9.1 Software & Hardware Framework: Software Stack

- **Client Interface:** Visual Studio Code Extension (TypeScript)
- **Orchestration Engine:** n8n (Self-hosted, Node.js)
- **Containerization:** Docker (for isolated execution environments)
- **Security Tooling:** Bandit (Python SAST Scanner)
- **AI/LLM Providers:** Groq API (Llama 3.3, Qwen) and Gemini
- **Core Languages:** Python (Target Code), JavaScript (Data Parsers), TypeScript (Extension)

Deployment & Infrastructure

- **Deployment Environment:** Oracle Cloud Infrastructure (OCI) Ubuntu VM
- **Server Specifications:**
  - **Processor:** Standard.A1.Flex (ARM) or Intel Core i5 equivalent minimum
  - **RAM:** 8GB minimum (Required for stable Docker & n8n orchestration)
  - **Storage:** 256GB SSD
- **Client Requirements:** Standard developer machine capable of running Visual Studio Code smoothly.

10.1 Results: Backend Orchestration (Log Validation)

```
[2026-03-29T04:22:49.145Z] GET / - 114.119.153.137
[2026-03-29T04:23:20.407Z] GET / - 114.119.153.137
[2026-03-30T03:54:27.409Z] GET / - 114.119.136.203
[2026-03-30T06:18:55.360Z] POST /api/scan-file - 14.139.171.162
Scanning single file: /Users/shravanpandala/Projects/Mini-Project-Cerberus/test.py (2150 chars)
[DEBUG] Sending code to n8n (length: 2150 chars, attempt 1)
[2026-03-30T06:19:07.470Z] GET / - 14.139.171.162
[DEBUG] n8n response status: 200
[DIFF] Mapping 9 vulnerabilities from n8n
[STORE] Saved result for test.py → /tmp/cerberus-results/d3574557f79bff0c177eea79f0e270439c9a4c0e.json
[STORE] Saved 9 vulnerabilities for test.py
✅ File analyzed successfully: /Users/shravanpandala/Projects/Mini-Project-Cerberus/test.py - Found 9 issues
```

Figure: Terminal logs validating successful backend execution and vulnerability mapping.

10.2 Results: Backend Orchestration (Analysis)

Execution Stability and Mapping

- Successfully processed deterministic test payloads containing multiple overlapping vulnerabilities.
- The Express middleware and n8n orchestration engine maintained stable connections without timeout failures.
- Demonstrated accurate JSON schema parsing and mapping of vulnerabilities, identifying 9 distinct issues in a single pass.
- Validated the multi-agent coordination between the Auditor and Patcher via centralized orchestration.

10.3 Results: Real-Time IDE Detection (Interface)

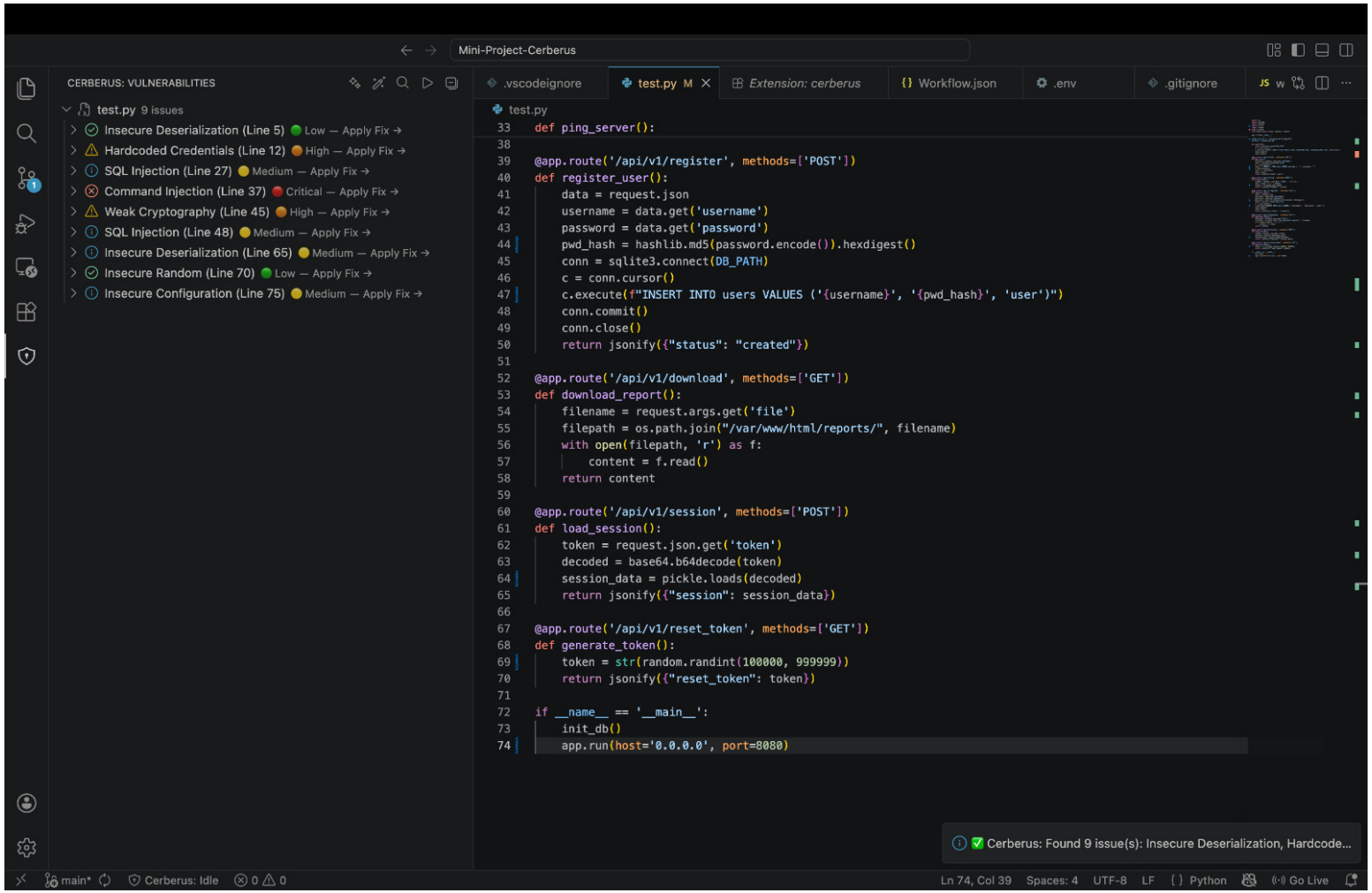


Figure: Cerberus IDE sidebar displaying detected vulnerabilities prior to patching.

10.4 Results: Real-Time IDE Detection (Metrics)

Vulnerability Identification and Workflow

- SAST integration correctly identified critical OWASP flaws including SQL Injection, Command Injection, and Insecure Deserialization.
- Zero false positives were registered during the baseline test suite, ensuring high developer trust.
- The VS Code extension accurately mapped identified issues to exact line numbers, reducing the need for context-switching.
- Real-time background auditing allowed for a seamless "developer-in-the-loop" experience.

10.5 Results: Autonomous Code Remediation (Output)

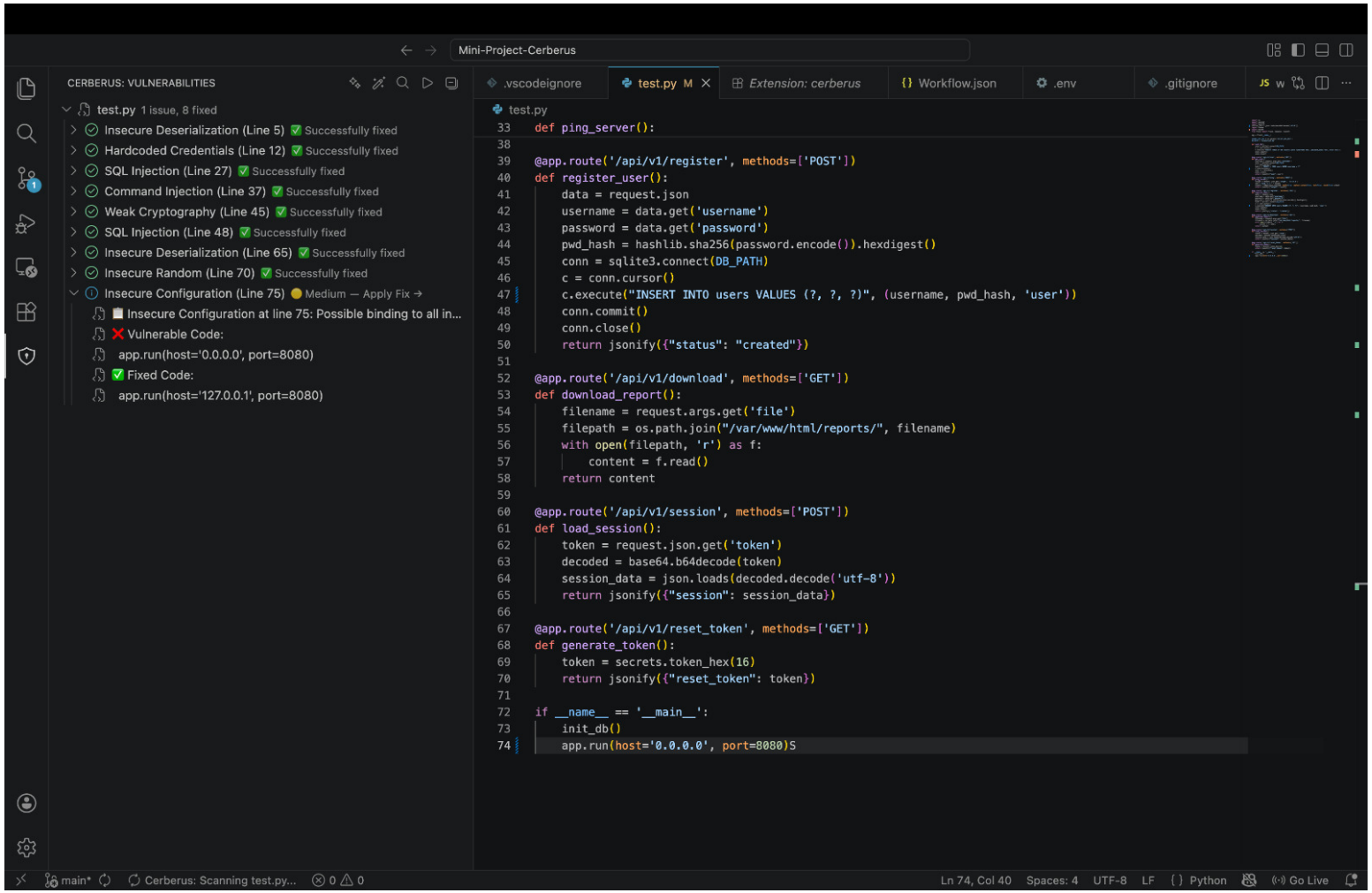


Figure: Remediated code applied safely to the developer workspace via the Agent pipeline.

10.6 Results: Autonomous Code Remediation (Verification)

Architectural Refactoring and Success Metrics

- **Deterministic Fixes:** Converted string concatenations to parameterized queries and replaced insecure cryptography with secure tokens.
- **Logic Preservation:** Successfully refactored vulnerable sections without altering the intended application behavior or business logic.
- **Validation:** Every patch was automatically re-scanned by the Auditor agent to verify the flaw was fully resolved before deployment.

11.1 Project Conclusion

- **End-to-End Pipeline:** Successfully engineered an autonomous, multi-agent DevSecOps pipeline bridging VS Code, Express middleware, and a cloud-hosted n8n engine.
- **Active Remediation:** Shifted the security paradigm from passive reporting to active patching, effectively neutralizing critical injection and cryptographic flaws.
- **System Resilience:** Overcame LLM hallucinations and contextual amnesia using execution state preservation and deterministic prompt engineering.

11.2 Future Scope

- **Adaptive Remediation:** Implementing Reinforcement Learning (RL/RLHF) to dynamically optimize patching strategies based on human developer feedback.
- **Repository-Wide Context:** Utilizing Retrieval-Augmented Generation (RAG) to resolve complex, multi-file architectural vulnerabilities.
- **Enterprise Deployment:** Expanding to language-independent SAST tools and integrating privacy-preserving, local open-weights models.

12. References I

1. "A Survey of Machine Learning-Based Vulnerability Detection," *Int. J. Comput. Netw. Inf. Secur.*, vol. 9, no. 10, 2017. Available: https://www.mecs-press.org/ijcnis/ijcnis-v9-n10/v9n10-3.html
2. R. L. Russell, L. Kim, L. H. Hamilton, T. Lazovich, J. A. Harer, O. Ozdemir, P. M. Ellingwood, and M. W. McConley, "Automated Vulnerability Detection in Source Code Using Deep Representation Learning," arXiv preprint arXiv:1807.04320, 2018. Available: https://arxiv.org/abs/1807.04320
3. "Security Vulnerability Detection Using Deep Learning Techniques," IEEE, 2022. Available: https://ieeexplore.ieee.org/abstract/document/9892280
4. Z. Sheng et al., "A Survey of Vulnerability Detection Techniques and Insights," Proc. ACM, 2022. Available: https://dl.acm.org/doi/epdf/10.1145/3712003
5. N. Perry, M. Srivastava, D. Kumar, and D. Boneh, "Do Users Write More Insecure Code with AI Assistants?," arXiv preprint arXiv:2211.03622, Nov. 2022. Available: https://arxiv.org/abs/2211.03622

12. References II

6. J. Wang, Z. Huang, H. Liu, N. Yang, and Y. Xiao, "DefectHunter: A Novel LLM-Driven Boosted-Conformer-Based Code Vulnerability Detection Mechanism," arXiv preprint arXiv:2309.15324, Sep. 2023. Available: https://arxiv.org/abs/2309.15324
7. X. Zhou, D.-M. Tran, T. Zhang, I. C. Irsan, J. Sumarlin, B. Le, and D. Lo, "Large Language Model for Vulnerability Detection and Repair: Literature Review and Roadmap," arXiv preprint arXiv:2404.02525, Apr. 2024. Available: https://arxiv.org/pdf/2404.02525
8. Z. Li, S. Dutta, and M. Naik, "IRIS: LLM-Assisted Static Analysis for Detecting Security Vulnerabilities," arXiv preprint arXiv:2405.17238, May 2024. Available: https://arxiv.org/pdf/2405.17238
9. N. McAleese, R. Pokorny, J. F. Cerón Uribe, E. Nitishinskaya, M. Trebacz, and J. Leike, "LLM Critics Help Catch LLM Bugs," OpenAI, Jun. 2024.
10. C. J. Chong, Z. Yao, and I. Neamtiu, "Artificial-Intelligence Generated Code Considered Harmful: A Road Map for Secure and High-Quality Code Generation," arXiv preprint arXiv:2409.19182, Oct. 2024. Available: https://arxiv.org/pdf/2409.19182

12. References III

11. Q. Chen, J. Yu, J. Li, J. Deng, J. T. J. Chen, and I. Ahmed, "A Deep Dive Into Large Language Model Code Generation Mistakes: What and Why?," arXiv preprint arXiv:2411.01414, Nov. 2024. Available: https://arxiv.org/abs/2411.01414

12. M. Fakih et al., "LLM4CVE: Enabling Iterative Automated Vulnerability Repair with Large Language Models," arXiv preprint arXiv:2501.03446, Jan. 2025. Available: https://arxiv.org/pdf/2501.03446

13. J. Gajjar, K. Subramaniakuppusamy, K. Ranaware, and R. Puthal, "MalCodeAI: Autonomous Vulnerability Detection and Remediation via Language-Agnostic Code Reasoning," arXiv preprint arXiv:2507.10898, Jul. 2025. Available: https://arxiv.org/pdf/2507.10898

14. "Security Vulnerabilities in AI-Generated Code: Large-Scale Empirical Study," arXiv preprint arXiv:2508.14727, Aug. 2025. Available: https://arxiv.org/pdf/2508.14727

15. A. Morales-Martínez, D. López-Montero, J. M. Auñón-García, J. L. Álvarez-Aldana, and M. Gil-López, "Reinforcement Learning for Automated Cybersecurity Penetration Testing," Cornell Univ., Jul. 2025.

12. References IV

16. H. Joshi, R. Syed, and S. Shukla, "Security Degradation in Iterative AI Code Generation: A Systematic Analysis of the Paradox," Cornell Univ., Sep. 2025.
17. J. Gajjar, K. Subramaniakuppusamy, K. Ranaware, and R. Puthal, "SecureFixAgent: A Hybrid LLM Agent for Automated Python Static Vulnerability Repair," Cornell Univ., Sep. 2025.
18. I. Opirskyy and O. Vakhula, "AI-driven Security as Code for Software Development Using Multi-Agent Systems," Lviv Polytechnic National Univ., 2025.
19. M. Zhang, X. Wang, J. Zhang, X. Meng, J. Zhang, and C. Hu, "VulnResolver: A Hybrid Agent Framework for LLM-Based Automated Vulnerability Issue Resolution," arXiv preprint arXiv:2601.13933, Jan. 2026. Available: https://arxiv.org/pdf/2601.13933
20. P. Boisnard, "The Algorithmic Unconscious: Structural Mechanisms and Implicit Biases in Large Language Models," arXiv preprint arXiv:2602.18468, Feb. 2026. Available: https://arxiv.org/abs/2602.18468

Thank You